

UNIT 2: Introduction to C Programming Language

Complete Notes for Unit 2 - C basics

Topics: Flowchart, Algorithm, Structure of C, Data Types, Operators, Control Structures

Contents

1	History and Evolution of C Language	4
1.1	Development Timeline	4
1.2	Predecessor Languages	4
1.3	Evolution and UNIX Connection	4
2	Characteristics of C Language	4
3	Structure of C Program	4
3.1	Sections Explained	5
4	Character Set in C	5
4.1	Categories	5
5	Keywords and Identifiers	6
5.1	Keywords (32 in C)	6
5.2	Identifiers	6
6	Data Types in C	7
6.1	Classification	7
6.2	Basic/Primary Data Types	7
6.3	Type Modifiers	7
7	Constants in C	7
7.1	Types of Constants	7
7.1.1	1. Numeric Constants	7
7.1.2	2. Character Constants	8
7.1.3	3. String Constants	8
7.2	Defining Symbolic Constants	8
8	Variables in C	8
8.1	Variable Declaration	8
8.2	Scope of Variables	9
9	Storage Classes	9
9.1	1. auto (Automatic)	9
9.2	2. register	9
9.3	3. static	9
9.4	4. extern	9

10 Operators in C	10
10.1 Classification of Operators	10
10.2 1. Arithmetic Operators	10
10.3 2. Relational Operators	10
10.4 3. Logical Operators	10
10.5 4. Assignment Operators	11
10.6 5. Increment/Decrement Operators	11
10.7 6. Conditional (Ternary) Operator	11
10.8 7. Bitwise Operators	11
10.9 8. Special Operators	11
11 Operator Precedence and Associativity	12
12 Expressions in C	12
12.1 Types of Expressions	12
13 Type Conversion and Type Casting	12
13.1 Type Conversion (Implicit)	12
13.2 Type Casting (Explicit)	13
14 Preprocessor Directives	13
14.1 Types	13
14.1.1 1. File Inclusion: #include	13
14.1.2 2. Macro Definition: #define	13
14.1.3 3. Conditional Compilation	13
15 Decision Making and Branching	13
15.1 1. if Statement	13
15.2 2. if-else Statement	13
15.3 3. if-else-if Ladder	14
15.4 4. Nested if	14
15.5 5. switch-case Statement	14
16 Loop Control Statements	14
16.1 1. break Statement	14
16.2 2. continue Statement	14
16.3 3. default in switch	15
16.4 4. exit() Function	15
17 Programming Errors	15
17.1 Types of Errors	15
17.1.1 1. Syntax Errors	15
17.1.2 2. Runtime Errors	15
17.1.3 3. Logical Errors	15
18 Flowchart and Algorithm	15
18.1 Algorithm	15
18.2 Flowchart	16
19 Memory Representation	16
19.1 Integer Representation	16
19.2 Character Representation	16
19.3 Floating Point	16

20 Important Programs	16
20.1 Program 1: Welcome Message	16
20.2 Program 2: Add Two Numbers	16
20.3 Program 3: Area of Circle	17
20.4 Program 4: Celsius to Fahrenheit	17
20.5 Program 5: All Arithmetic Operators	17
20.6 Program 6: Evaluate Expression	17
20.7 Program 7: Largest of Two (Ternary)	18
20.8 Program 8: Increment/Decrement	18
20.9 Program 9: All Data Types	18
20.10 Program 10: Swap Without Temp	18
20.11 Program 11: ASCII Value	19
20.12 Program 12: Integer vs Float Division	19
20.13 Program 13: Relational and Logical Operators	19
20.14 Program 14: Macro for Square	19
20.15 Program 15: sizeof Operator	19
21 Quick Revision Points	20
22 Common Exam Questions - Quick Answers	20
22.1 1 Mark Answers	20
22.2 2 Mark Key Points	21
22.3 3 Mark Explanations	21
22.4 Fill in the Blanks - Answers	22
23 Important Formulas and Concepts	22
23.1 Expression Evaluation Examples	22
23.2 Type Conversion Examples	23
24 Exam Tips and Strategies	24

1 History and Evolution of C Language

1.1 Development Timeline

- **Developer:** Dennis Ritchie at Bell Laboratories (AT&T)
- **Year:** 1972
- **Purpose:** Originally developed for writing UNIX operating system

1.2 Predecessor Languages

1. **BCPL (1967)** - Developed by Martin Richards
2. **B Language (1970)** - Developed by Ken Thompson
3. **C Language (1972)** - Improved version combining features of both

1.3 Evolution and UNIX Connection

- UNIX was initially written in assembly language
- Dennis Ritchie developed C to rewrite UNIX in a higher-level language
- By 1973, most of UNIX was rewritten in C
- This made UNIX portable across different hardware platforms
- C became the standard for system programming

2 Characteristics of C Language

Key Features - Remember These!

1. **Middle-Level Language:** Combines features of high-level and low-level languages
2. **Structured Language:** Supports modular programming using functions
3. **Portable:** Programs can run on different machines with little/no modification
4. **Efficient:** Fast execution, direct memory access
5. **Rich Library:** Built-in functions for various operations
6. **Modularity:** Programs can be divided into functions
7. **Case Sensitive:** `Variable` and `variable` are different
8. **Extensible:** Can add own functions to C library
9. **Pointer Support:** Direct memory manipulation
10. **Recursion:** Functions can call themselves

3 Structure of C Program

```

/* Documentation Section - Comments about program */
// Preprocessor Directives Section
#include <stdio.h>
#define PI 3.14

// Global Declaration Section
int global_var;

// Function Prototype Declaration
void myFunction();

// Main Function - Entry point of program
int main() {
    // Local Variable Declaration
    int a, b, sum;

    // Executable Statements
    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);
    sum = a + b;
    printf("Sum = %d", sum);

    return 0;
}

// User-defined Function Definition
void myFunction() {
    // Function body
}

```

3.1 Sections Explained

1. **Documentation:** Comments explaining program purpose
2. **Preprocessor Directives:** Instructions processed before compilation
3. **Global Declaration:** Variables/functions visible to all functions
4. **main() Function:** Program execution starts here (mandatory)
5. **User-defined Functions:** Additional functions created by programmer

4 Character Set in C

Character Set

The set of valid characters recognized by C compiler

4.1 Categories

1. **Letters:** A-Z, a-z (52 letters)
2. **Digits:** 0-9 (10 digits)
3. **Special Characters:**
 - Arithmetic: + - * / %
 - Relational: < > = !

- Logical: && — !
- Punctuation: , ; : . ?
- Brackets: () [] { }
- Others: # \$ @ ~ ' ' " \ — _

4. **White Spaces:** blank space, tab, newline, carriage return

5 Keywords and Identifiers

5.1 Keywords (32 in C)

All C Keywords - Must Memorize!

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Properties:

- Reserved words - cannot be used as identifiers
- Always written in lowercase
- Have predefined meaning in C

5.2 Identifiers

Identifier

Names given to variables, functions, arrays, structures, etc.

Rules for Naming Identifiers:

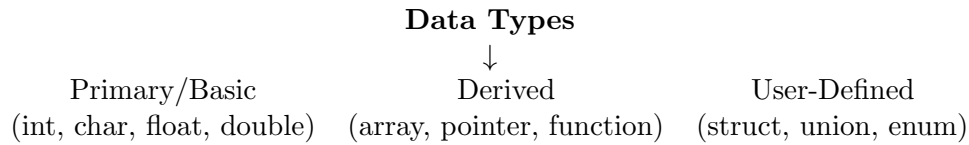
1. Must begin with letter (A-Z, a-z) or underscore (_)
2. Can contain letters, digits (0-9), underscore
3. Cannot use keywords
4. Case sensitive (`sum` $\neq$ `Sum`)
5. No special characters except underscore
6. No spaces allowed
7. Maximum length typically 31 characters

Examples:

- Valid: `sum`, `_value`, `student1`, `Total_Marks`
- Invalid: `1student`, `int`, `my-var`, `total marks`

6 Data Types in C

6.1 Classification



6.2 Basic/Primary Data Types

Data Type	Size	Range	Format
char	1 byte	-128 to 127	%c
unsigned char	1 byte	0 to 255	%c
int	2 or 4 bytes	-32768 to 32767	%d
unsigned int	2 or 4 bytes	0 to 65535	%u
short int	2 bytes	-32768 to 32767	%hd
long int	4 bytes	-2,147,483,648 to 2,147,483,647	%ld
float	4 bytes	3.4E-38 to 3.4E+38	%f
double	8 bytes	1.7E-308 to 1.7E+308	%lf
long double	10 bytes	Extended precision	%Lf
void	0 bytes	No value	-

Table 1: Data Types with Size and Range

6.3 Type Modifiers

- **signed:** Can store negative and positive values (default)
- **unsigned:** Only positive values
- **short:** Reduces size
- **long:** Increases size

7 Constants in C

Constant

Fixed value that cannot be changed during program execution

7.1 Types of Constants

7.1.1 1. Numeric Constants

- **Integer Constants:** Whole numbers without decimal point
 - Decimal: 123, -456
 - Octal: 0123 (starts with 0)
 - Hexadecimal: 0x123 (starts with 0x)
- **Floating Constants:** Numbers with decimal point
 - Examples: 3.14, -0.5, 2.5e10

7.1.2 2. Character Constants

- Single character enclosed in single quotes
- Examples: 'A', 'z', '5', '\$'
- **Escape Sequences:**
 - \n - newline
 - \t - tab
 - \b - backspace
 - \r - carriage return
 - \\ - backslash
 - \' - single quote
 - \" - double quote
 - \0 - null character

7.1.3 3. String Constants

- Sequence of characters enclosed in double quotes
- Examples: "Hello", "C Programming"
- Automatically null-terminated

7.2 Defining Symbolic Constants

Method 1: Using #define

```
#define PI 3.14159
#define MAX 100
```

Method 2: Using const keyword

```
const float PI = 3.14159;
const int MAX = 100;
```

8 Variables in C

Variable

Named memory location whose value can change during program execution

8.1 Variable Declaration

Syntax: data_type variable_name;

```
int age;
float salary, tax;
char grade = 'A';
```


8.2 Scope of Variables

1. Local Variables:

- Declared inside a function or block
- Accessible only within that function/block
- Created when function is called, destroyed when exits

2. Global Variables:

- Declared outside all functions
- Accessible to all functions in program
- Exist throughout program execution

9 Storage Classes

Four Storage Classes in C

1. auto - Default for local variables
2. register - Stored in CPU register
3. static - Retains value between function calls
4. extern - Declares global variable from another file

9.1 1. auto (Automatic)

- Default storage class for local variables
- Scope: Local to block
- Lifetime: Until block execution
- Initial value: Garbage

9.2 2. register

- Stored in CPU register instead of RAM
- Faster access
- Cannot use address operator (&)

9.3 3. static

- Retains value between function calls
- Initialized only once
- Initial value: 0 (automatically)

9.4 4. extern

- Used to declare global variable in another file
- Provides visibility to global variable

10 Operators in C

10.1 Classification of Operators

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Increment/Decrement Operators
6. Conditional/Ternary Operator
7. Bitwise Operators
8. Special Operators

10.2 1. Arithmetic Operators

Operator	Operation	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b

10.3 2. Relational Operators

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

10.4 3. Logical Operators

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

Operator	Equivalent
<code>+=</code>	<code>a = a + b</code>
<code>-=</code>	<code>a = a - b</code>
<code>*=</code>	<code>a = a * b</code>
<code>/=</code>	<code>a = a / b</code>

10.5 4. Assignment Operators

10.6 5. Increment/Decrement Operators

Pre vs Post

- **++a (Pre-increment):** Increment first, then use
- **a++ (Post-increment):** Use first, then increment
- **--a (Pre-decrement):** Decrement first, then use
- **a-- (Post-decrement):** Use first, then decrement

10.7 6. Conditional (Ternary) Operator

Syntax: `condition ? expression1 : expression2`

```
max = (a > b) ? a : b;
```

10.8 7. Bitwise Operators

Operator	Operation
<code>&</code>	Bitwise AND
<code> </code>	Bitwise OR
<code>^</code>	Bitwise XOR
<code>~</code>	Bitwise NOT
<code><<</code>	Left shift
<code>>></code>	Right shift

10.9 8. Special Operators

1. **sizeof():** Returns size in bytes
2. **Comma (,):** Separates expressions
3. **Address (&):** Returns address of variable
4. **Pointer (*):** Pointer operations

11 Operator Precedence and Associativity

Operator Precedence (High to Low)

1. Parentheses: `()`
2. Unary: `++`, `-`, `!`, `sizeof`
3. Multiplicative: `*`, `/`, `%`
4. Additive: `+`, `-`
5. Shift: `<<`, `>>`
6. Relational: `<`, `<=`, `>`, `>=`
7. Equality: `==`, `!=`
8. Bitwise AND: `&`
9. Bitwise XOR: `^`
10. Bitwise OR: `|`
11. Logical AND: `&&`
12. Logical OR: `||`
13. Conditional: `?:`
14. Assignment: `=`, `+=`, `-=`
15. Comma: `,`

12 Expressions in C

Expression

Combination of operators, constants, and variables that evaluates to a single value

12.1 Types of Expressions

1. **Arithmetic Expression:** `a + b * c`
2. **Relational Expression:** `a < b`
3. **Logical Expression:** `(a < b) && (c < d)`
4. **Assignment Expression:** `a = b + c`
5. **Conditional Expression:** `(a < b) ? a : b`

13 Type Conversion and Type Casting

13.1 Type Conversion (Implicit)

- Automatic conversion by compiler
- Lower type converted to higher type

13.2 Type Casting (Explicit)

- Programmer-forced conversion
- **Syntax:** (type) expression

```
float a = 5.8;  
int b = (int) a;    // b = 5
```

14 Preprocessor Directives

Preprocessor

Program that processes source code before compilation

14.1 Types

14.1.1 1. File Inclusion: #include

```
#include <stdio.h>  
#include "myfile.h"
```

14.1.2 2. Macro Definition: #define

```
#define PI 3.14159  
#define MAX 100  
#define SQUARE(x) ((x) * (x))
```

14.1.3 3. Conditional Compilation

```
#ifdef DEBUG  
    printf("Debug mode");  
#endif  
  
#ifndef PI  
    #define PI 3.14  
#endif
```

15 Decision Making and Branching

15.1 1. if Statement

```
if (condition) {  
    // statements  
}
```

15.2 2. if-else Statement

```
if (condition) {  
    // true statements  
} else {  
    // false statements  
}
```

15.3 3. if-else-if Ladder

```
if (condition1) {  
    // statements  
} else if (condition2) {  
    // statements  
} else {  
    // default  
}
```

15.4 4. Nested if

```
if (condition1) {  
    if (condition2) {  
        // statements  
    }  
}
```

15.5 5. switch-case Statement

switch-case Rules

- Tests single variable/expression
- Only integer or character values
- Each case must have unique constant
- break statement exits switch
- default case is optional

```
switch (expression) {  
    case constant1:  
        // statements  
        break;  
    case constant2:  
        // statements  
        break;  
    default:  
        // default statements  
}
```

16 Loop Control Statements

16.1 1. break Statement

- Terminates loop or switch immediately
- Control passes to statement after loop

16.2 2. continue Statement

- Skips current iteration
- Continues with next iteration

16.3 3. default in switch

- Optional case in switch
- Executes when no case matches

16.4 4. exit() Function

- Terminates entire program
- exit(0) - normal termination
- exit(1) - abnormal termination

17 Programming Errors

17.1 Types of Errors

17.1.1 1. Syntax Errors

- Errors in language rules
- Detected by compiler
- Examples: Missing semicolon, undeclared variables

17.1.2 2. Runtime Errors

- Occur during program execution
- Examples: Division by zero, array index out of bounds

17.1.3 3. Logical Errors

- Program runs but produces wrong output
- Examples: Wrong formula, incorrect logic

18 Flowchart and Algorithm

18.1 Algorithm

Algorithm

Step-by-step procedure to solve a problem

Characteristics:

- Finite - must terminate
- Definite - each step clear
- Input - zero or more inputs
- Output - at least one output
- Effectiveness - operations basic

18.2 Flowchart

Flowchart

Pictorial representation of algorithm using symbols

Symbols:

- Oval: Start/Stop
- Parallelogram: Input/Output
- Rectangle: Process
- Diamond: Decision
- Arrow: Flow direction

19 Memory Representation

19.1 Integer Representation

- signed int: Uses 2's complement for negative
- Range: -32768 to +32767 (2 bytes)

19.2 Character Representation

- Stored using ASCII values
- 'A' = 65, 'a' = 97, '0' = 48

19.3 Floating Point

- IEEE 754 Standard
- float: Sign(1) + Exponent(8) + Mantissa(23)
- double: Sign(1) + Exponent(11) + Mantissa(52)

20 Important Programs

20.1 Program 1: Welcome Message

```
#include <stdio.h>
int main() {
    printf("Welcome to C Programming");
    return 0;
}
```

20.2 Program 2: Add Two Numbers

```
#include <stdio.h>
int main() {
    int a, b, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d", sum);
}
```



```
    return 0;
}
```

20.3 Program 3: Area of Circle

```
#include <stdio.h>
#define PI 3.14159
int main() {
    float radius, area;
    printf("Enter radius: ");
    scanf("%f", &radius);
    area = PI * radius * radius;
    printf("Area = %.2f", area);
    return 0;
}
```

20.4 Program 4: Celsius to Fahrenheit

```
#include <stdio.h>
int main() {
    float c, f;
    printf("Enter Celsius: ");
    scanf("%f", &c);
    f = (c * 9/5) + 32;
    printf("Fahrenheit = %.2f", f);
    return 0;
}
```

20.5 Program 5: All Arithmetic Operators

```
#include <stdio.h>
int main() {
    int a = 10, b = 3;
    printf("Addition: %d\n", a + b);
    printf("Subtraction: %d\n", a - b);
    printf("Multiplication: %d\n", a * b);
    printf("Division: %d\n", a / b);
    printf("Modulus: %d\n", a % b);
    return 0;
}
```

20.6 Program 6: Evaluate Expression

```
#include <stdio.h>
int main() {
    int a=10, b=5, c=3, d=2, e=1, x;
    x = a + b * c - d / e;
    printf("Result = %d", x);
    return 0;
}
```

20.7 Program 7: Largest of Two (Ternary)

```
#include <stdio.h>
int main() {
    int a, b, max;
    scanf("%d %d", &a, &b);
    max = (a > b) ? a : b;
    printf("Max = %d", max);
    return 0;
}
```

20.8 Program 8: Increment/Decrement

```
#include <stdio.h>
int main() {
    int a = 5, b;
    b = a++;
    printf("Post: b=%d, a=%d\n", b, a);
    a = 5;
    b = ++a;
    printf("Pre: b=%d, a=%d\n", b, a);
    return 0;
}
```

20.9 Program 9: All Data Types

```
#include <stdio.h>
int main() {
    int i = 10;
    float f = 3.14;
    char c = 'A';
    double d = 123.456;
    printf("Int: %d\n", i);
    printf("Float: %.2f\n", f);
    printf("Char: %c\n", c);
    printf("Double: %.3lf\n", d);
    return 0;
}
```

20.10 Program 10: Swap Without Temp

```
#include <stdio.h>
int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    printf("Before: a=%d, b=%d\n", a, b);
    a = a + b;
    b = a - b;
    a = a - b;
    printf("After: a=%d, b=%d\n", a, b);
    return 0;
}
```

20.11 Program 11: ASCII Value

```
#include <stdio.h>
int main() {
    char ch;
    printf("Enter character: ");
    scanf("%c", &ch);
    printf("ASCII = %d", ch);
    return 0;
}
```

20.12 Program 12: Integer vs Float Division

```
#include <stdio.h>
int main() {
    int a = 10, b = 3;
    float r1, r2;
    r1 = a / b;
    r2 = (float)a / b;
    printf("Int division: %.2f\n", r1);
    printf("Float division: %.2f\n", r2);
    return 0;
}
```

20.13 Program 13: Relational and Logical Operators

```
#include <stdio.h>
int main() {
    int a = 5, b = 10, c = 15;
    printf("a < b: %d\n", a < b);
    printf("a == b: %d\n", a == b);
    printf("(a < b) && (b < c): %d\n", (a < b) && (b < c));
    printf("(a > b) || (b < c): %d\n", (a > b) || (b < c));
    return 0;
}
```

20.14 Program 14: Macro for Square

```
#include <stdio.h>
#define SQUARE(x) ((x) * (x))
int main() {
    int n = 5;
    printf("Square of %d = %d", n, SQUARE(n));
    return 0;
}
```

20.15 Program 15: sizeof Operator

```
#include <stdio.h>
int main() {
    printf("Size of int: %d bytes\n", sizeof(int));
    printf("Size of float: %d bytes\n", sizeof(float));
    printf("Size of char: %d bytes\n", sizeof(char));
    printf("Size of double: %d bytes\n", sizeof(double));
    return 0;
}
```

21 Quick Revision Points

Must Remember for Exam

1. **C Developed:** Dennis Ritchie (1972) at Bell Labs
2. **Predecessors:** BCPL, B Language
3. **Keywords:** 32 reserved words
4. **Data Types:** int(4), char(1), float(4), double(8) bytes
5. **Storage Classes:** auto, register, static, extern
6. **Operators Precedence:** () $\hat{=}$ unary $\hat{=}$ * / % $\hat{=}$ + - $\hat{=}$ relational $\hat{=}$ logical $\hat{=}$ assignment
7. **Escape Sequences:** \n, \t, \b, \r, \0
8. **Preprocessor:** #include, #define, #ifdef, #ifndef
9. **Decision Making:** if, if-else, if-else-if, switch-case, nested if
10. **Loop Control:** break (exit loop), continue (skip iteration)
11. **Type Casting:** (type) expression
12. **Increment:** ++a (pre), a++ (post)
13. **Modulus:** Only with integers
14. **Logical:** && (AND), —— (OR), ! (NOT)
15. **Ternary:** condition ? true : false

22 Common Exam Questions - Quick Answers

22.1 1 Mark Answers

1. **Who developed C?** Dennis Ritchie
2. **Year?** 1972
3. **Characteristic?** Portable/Middle-level
4. **Keyword?** Reserved word with predefined meaning
5. **Data type example?** int, char, float
6. **Constant?** Fixed value that cannot change
7. **Identifier?** Name given to variable/function
8. **Assignment operator?** =
9. **sizeof?** Returns size in bytes
10. **Preprocessor directive?** #include, #define

22.2 2 Mark Key Points

1. **Two characteristics:** (1) Portable (2) Structured
2. **Character set:** Valid characters: letters, digits, special chars
3. **Keyword vs Identifier:** Keywords reserved, identifiers user-defined
4. **Integer vs Float:** Integer no decimal (10), Float has decimal (10.5)
5. **Variable declaration:** data_type name; Example: int age;
6. **Assignment:** = assigns value: a = 5;
7. **Unary vs Binary:** Unary one operand (++a), Binary two (a+b)
8. **Expression:** Combination: a + b * c
9. **#include:** Includes header: #include <stdio.h>
10. **Arithmetic:** + (add), - (subtract), * (multiply), / (divide), % (modulus)

22.3 3 Mark Explanations

1. **History of C:** C evolved from BCPL (1967) and B (1970). Dennis Ritchie developed C in 1972 at Bell Labs to rewrite UNIX. By 1973, UNIX was rewritten in C, making it portable.

2. **Three features:**

- Portable: Runs on different machines
- Efficient: Fast execution with low-level access
- Structured: Modular programming with functions

3. **Keywords and Identifiers:** Keywords are reserved words (32 in C) with predefined meaning like int, for, while. Identifiers are user-defined names for variables/functions following rules: start with letter/underscore, no spaces, no keywords.

4. **Types of constants:**

- Numeric: Integer (10, -5), Float (3.14, 2.5e10)
- Character: Single char in quotes ('A', '5')
- String: Characters in double quotes ("Hello")

5. **Variable naming rules:**

- Must start with letter or underscore
- Can contain letters, digits, underscore
- No keywords, no spaces
- Case sensitive
- Max 31 characters typically

6. **Data types with memory:**

- int: 4 bytes, stores whole numbers
- float: 4 bytes, stores decimal numbers (6 digits precision)
- char: 1 byte, stores single character

7. Operators with examples:

- Arithmetic: $a + b$, $a * b$
- Relational: $a < b$, $a == b$ (returns 0 or 1)
- Logical: $(a < b) \&\& (c < d)$, returns true/false

8. Operator precedence: Order of evaluation: Parentheses highest, then unary ($++$, $-$), then $*$, $/$, $\%$, then $+$, $-$, then relational, then logical, assignment lowest. Example: $a + b * c$ evaluates $b * c$ first.

9. Expression evaluation $a = b + c * d$:

- Step 1: $c * d$ (multiplication has higher precedence)
- Step 2: $b + \text{result}$ (addition)
- Step 3: $a = \text{final result}$ (assignment)

10. Preprocessor directives:

- `#define`: Defines symbolic constant/macro
- `#include`: Includes header file content
- Processed before compilation, start with `#`

22.4 Fill in the Blanks - Answers

Set 1: 1. Dennis Ritchie 2. 1972 3. BCPL, B 4. middle 5. operating

Set 2: 6. 32 7. reserved 8. letter, underscore 9. = 10. sizeof

Set 3: 11. declared 12. decimal 13. char 14. `#define` 15. const

Set 4: 16. `%` 17. higher 18. pre 19. relational 20. `*` (multiplication)

Set 5: 21. `#include` 22. `#define` 23. pre 24. symbolic 25. `#`

23 Important Formulas and Concepts**23.1 Expression Evaluation Examples**

Example 1: $x = 5 + 3 * 2$

- Step 1: $3 * 2 = 6$
- Step 2: $5 + 6 = 11$
- Result: $x = 11$

Example 2: $a = 10; b = ++a + a++;$

- $++a$ makes $a = 11$
- $11 + 11 = 22$ (a is 11 when used in $a++$)
- $a++$ makes $a = 12$ after evaluation
- Result: $b = 22$, $a = 12$

Example 3: $x = 10 \&\& 5 \&\& 3 \&\& 7$

- $10 \&\& 5$ evaluates to 1 (true)
- $3 \&\& 7$ evaluates to 1 (true)
- $1 \&\& 1 = 1$ (true)
- Result: $x = 1$

23.2 Type Conversion Examples

Implicit Conversion:

```
int a = 5;  
float b = 2.5;  
float result = a + b;  // a promoted to float, result = 7.5
```

Explicit Casting:

```
float a = 7.8;  
int b = (int)a;  // b = 7, decimal truncated  
int x = 10, y = 3;  
float z = (float)x / y;  // z = 3.333...
```

24 Exam Tips and Strategies

How to Score Full Marks

1. For 1-2 Mark Questions:

- Be precise and direct
- Give exact definitions
- Include examples where asked

2. For 3-4 Mark Questions:

- Start with definition
- Explain with 2-3 points
- Always give examples
- Draw diagrams if applicable

3. For 5-7 Mark Questions:

- Introduction/definition
- Detailed explanation with points
- Multiple examples
- Comparison tables if asked
- Diagrams/flowcharts
- Conclusion

4. For Programming Questions:

- Always include `#include <stdio.h>`
- Declare all variables
- Use proper indentation
- Add comments
- Include `return 0;`

5. Common Mistakes to Avoid:

- Don't confuse `=` with `==`
- Don't forget semicolons
- Don't mix up pre/post increment
- Don't forget `break` in switch
- Check operator precedence

All the Best for Your Exam!

Thank me Later!